

Food Before It Spoils

Redesigning the TSP for deadline-aware delivery

CS201 · Group [XX]

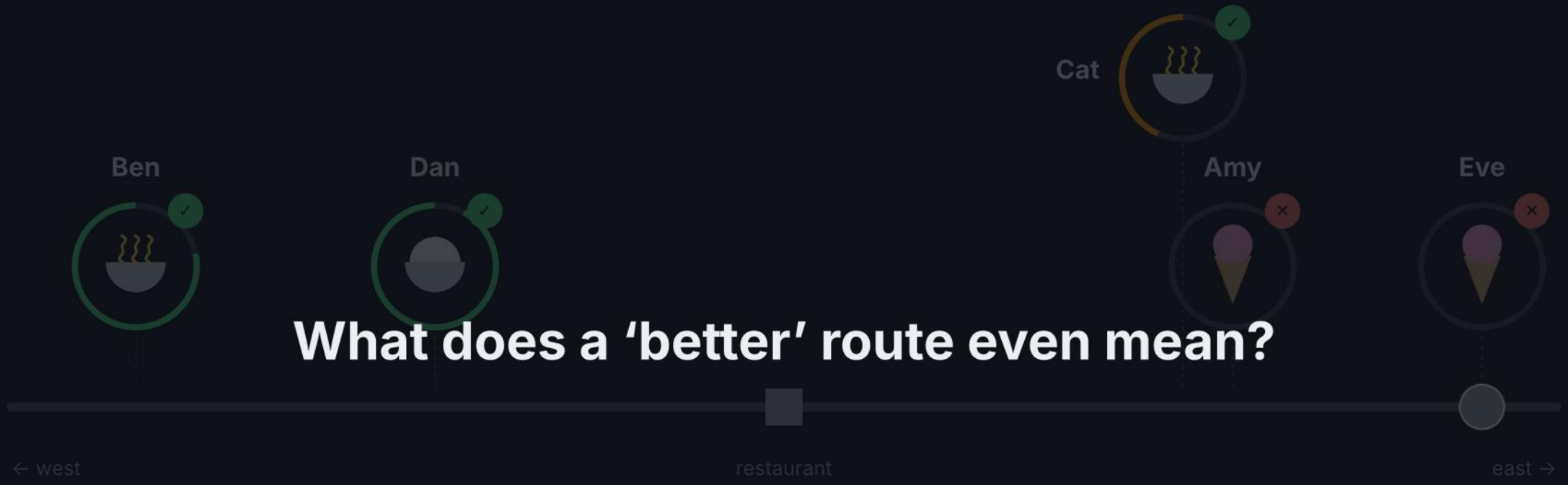
[Member names]

1

Problem

One rider, five orders, three shelf lives

The closest stop is not the right stop

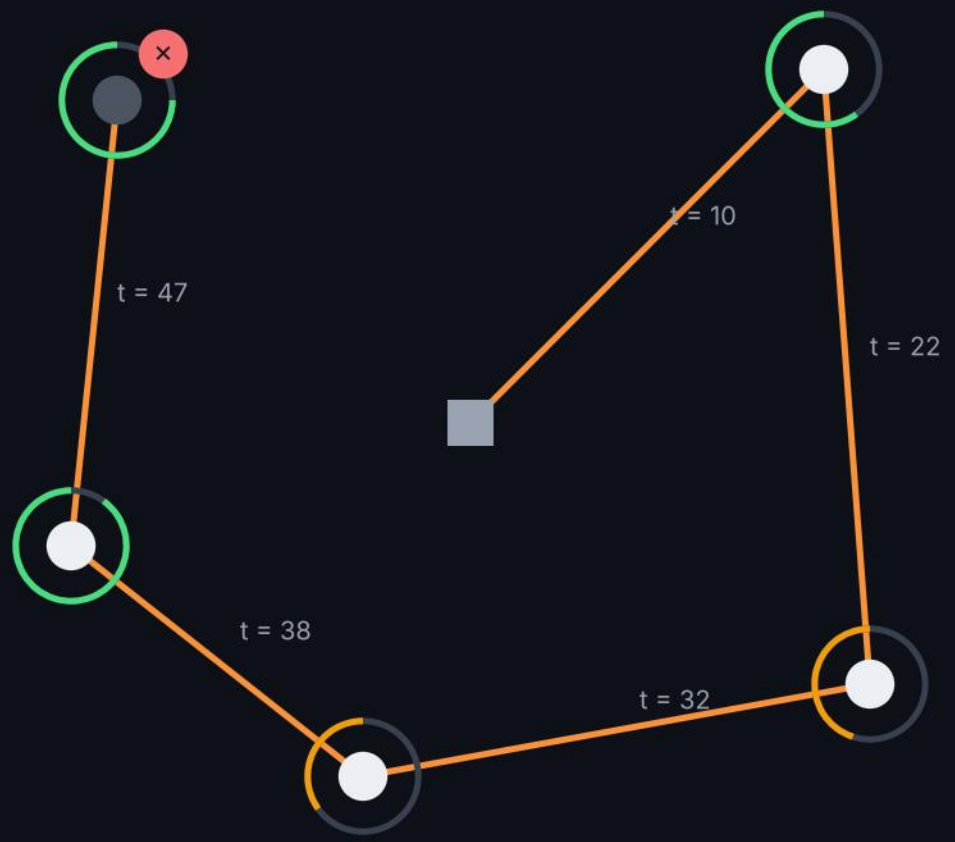


Delivered 3 of 5

Average wait 18.0 min

t = 40 min

Why the classic TSP is not enough



Goal: most on time, then least wait

Classic TSP		Our use case
Every city equally important	→	Each order has a deadline
Only the visit order matters	→	Arrival time matters
Must visit every city	→	Cancel orders that can't be on time
Tour returns to the start	→	Route ends at the last customer
Goal: shortest distance	→	Goal: most on time, then least wait

The Minimum Latency Problem with deadlines: NP-hard. We redesign heuristics.

The new problem

Input (known at t = 0)

Restaurant + n orders

Travel time w(i, j), minutes

Order time o, food, shelf life ℓ

Rider leaves at t = 0

Output

The order the rider visits

customers in.

Anyone left out is cancelled.

Rules

One rider, never returns

No time spent at the door

Wait = arrival a − order time o

Late = 0, always

Deadline d

V1 d = o + ℓ — food must not spoil

V2 d = o + min(ℓ, 60)

no one waits more than an hour

Goals, in this order

1. Fewest cancelled

2. Lowest average wait

Nearest neighbour

3 of 5

18.0 min

Total wait = Σa − Σo, and Σo is fixed: lowering the average wait means arriving earlier overall.

Reading the picture



position →
(minutes of driving)

time ↓

dashed line = still fresh
amber tick = deadline

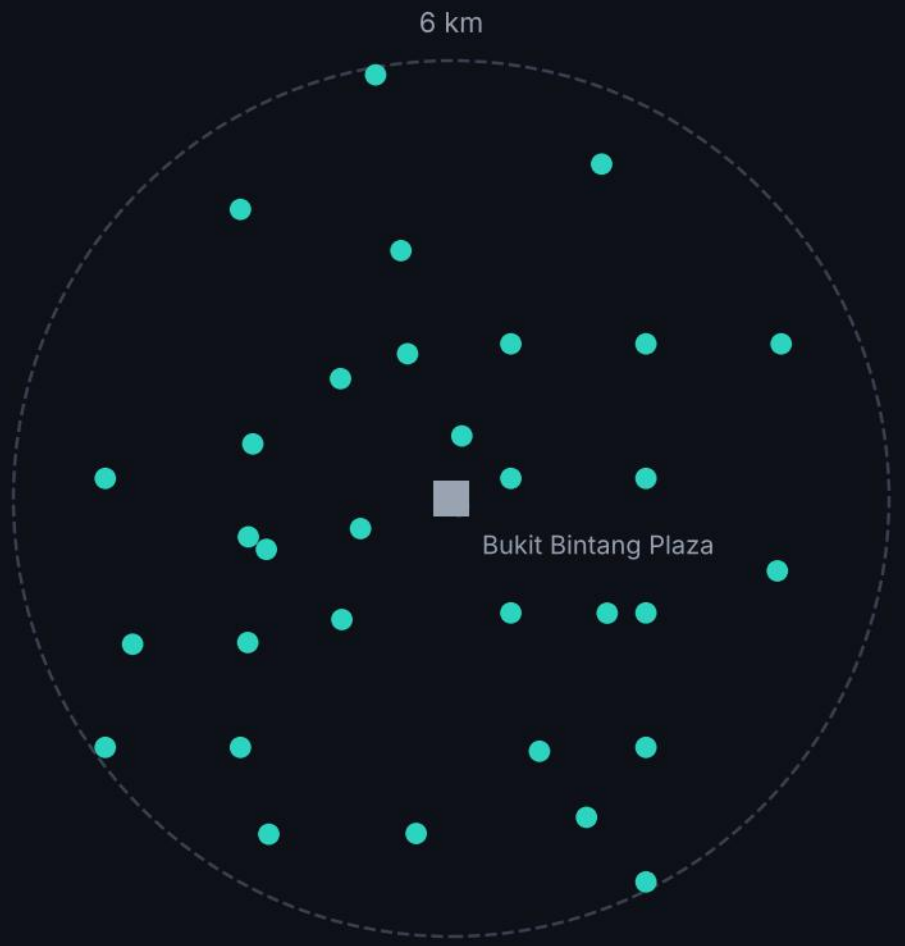
On time ⇔ the path hits the line above the tick.

The data

MY.txt: 71,709 rows

Class P within 6 km of Bukit Bintang Plaza: 40

Duplicate coordinates dropped: 33 left



Experiments

n =	5	8	10	20	50	100
×0.5	30	30	30	30	30	30
×1.0	30	30	30	30	30	30
×1.5	30	30	30	30	30	30

shelf-life tightness ↓
30 seeds per cell

The question



Approach A

Plan everything, then fix it

Approach B

Decide one stop at a time

In what order do we visit them, quickly, when deadlines conflict?

2

Two approaches

Plan first, or decide as you go

One fact both approaches rely on

$$t + w(c, v) > d_v$$

can't make it even driving straight there



$$a_v = t + w(c, x_1) + \cdots + w(x_k, v) \;\geq\; t + w(c, v) \;>\; d_v$$

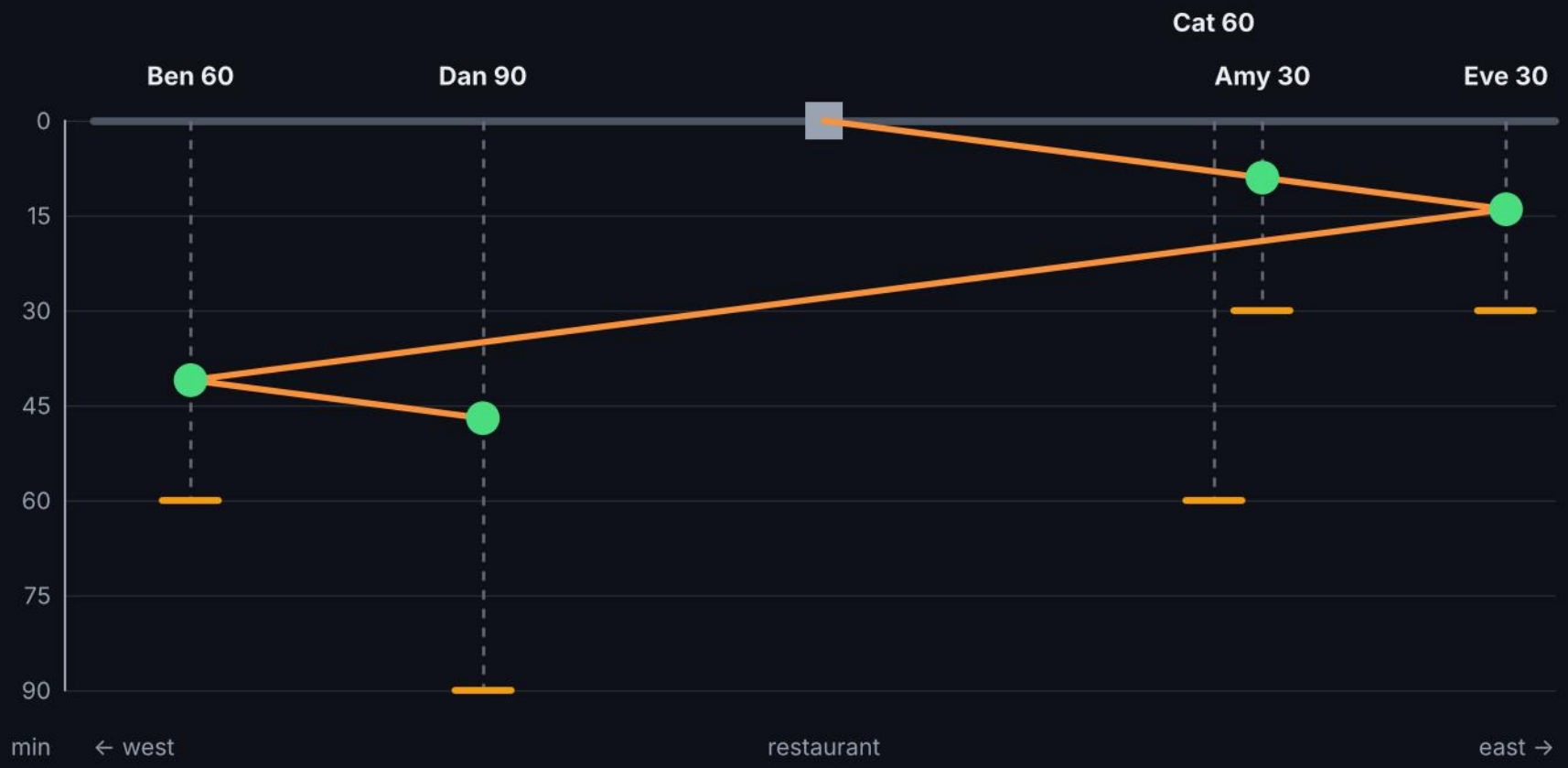
Cancelling is free. If anything went wrong, it went wrong earlier.

Approach A: the plan

Step 1: sort by deadline

Amy	Eve	Ben	Cat	Dan
d 30	d 30	d 60	d 60	d 90
9 ✓	14 ✓	41 ✓	62 × cancel	47 ✓

the plan =
list of all n customers



Delivered

4 of 5

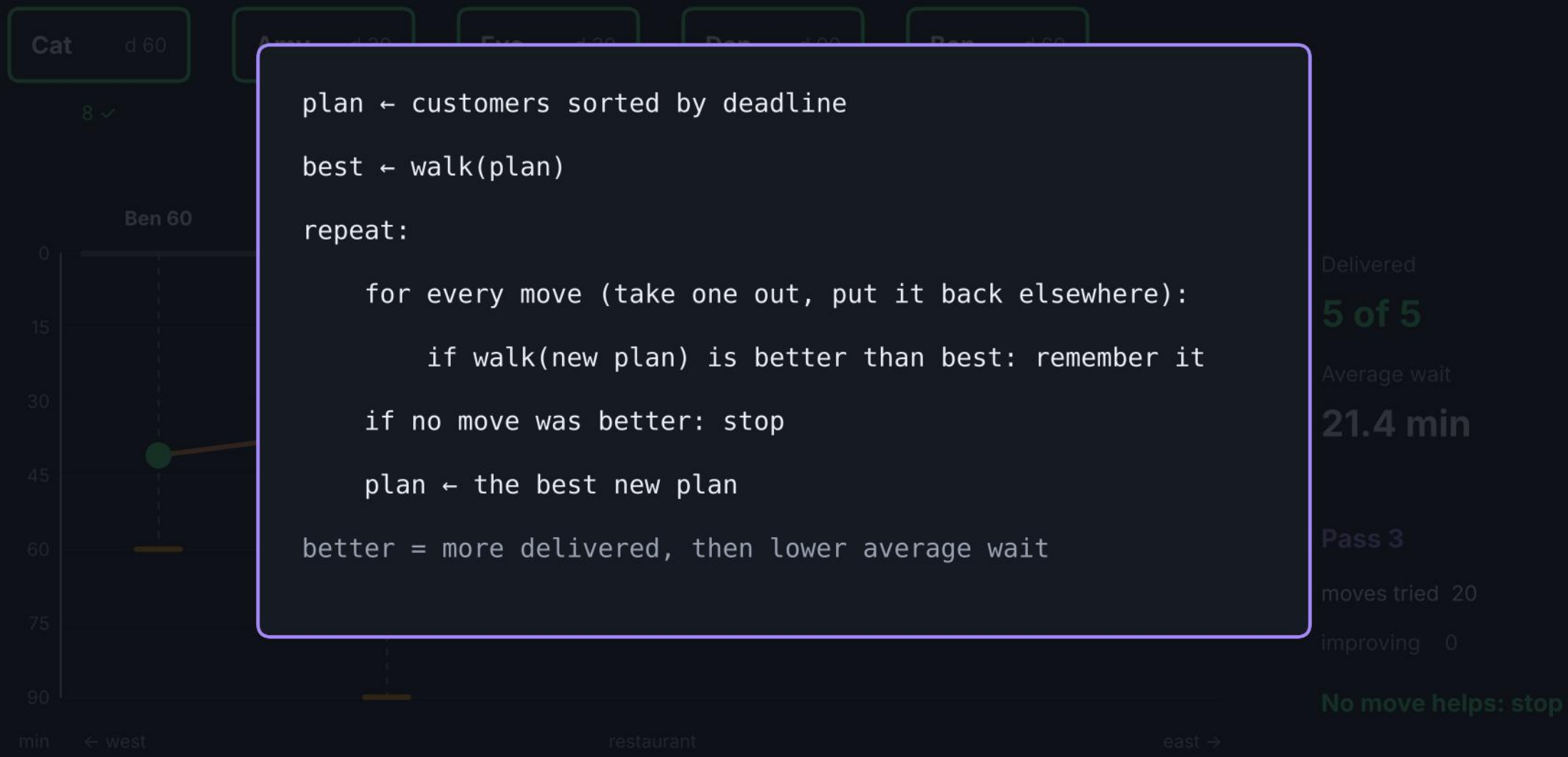
Average wait

27.75 min

- on time → drive there
- too late → cancel, stay put

Sorting by urgency ignores the map: Cat is lost.

Approach A: fixing the plan




```
plan ← customers sorted by deadline
best ← walk(plan)
repeat:
    for every move (take one out, put it back elsewhere):
        if walk(new plan) is better than best: remember it
    if no move was better: stop
    plan ← the best new plan
better = more delivered, then lower average wait
```

Every accepted move strictly improves the plan, so it always stops.

Approach B: one stop at a time



1. reachable on time · 5 left
2. fewest lost · 4 left
3. shortest travel · 1 left
4. least slack
5. lowest id

→ Dan, 7 min

visited[ ] · · · · ·

alive[ ] ✓ ✓ ✓ ✓ ✓

No heap: every score changes after every move.

Approach B in action



Approach B

Delivered

5 of 5

Average wait

27.0 min

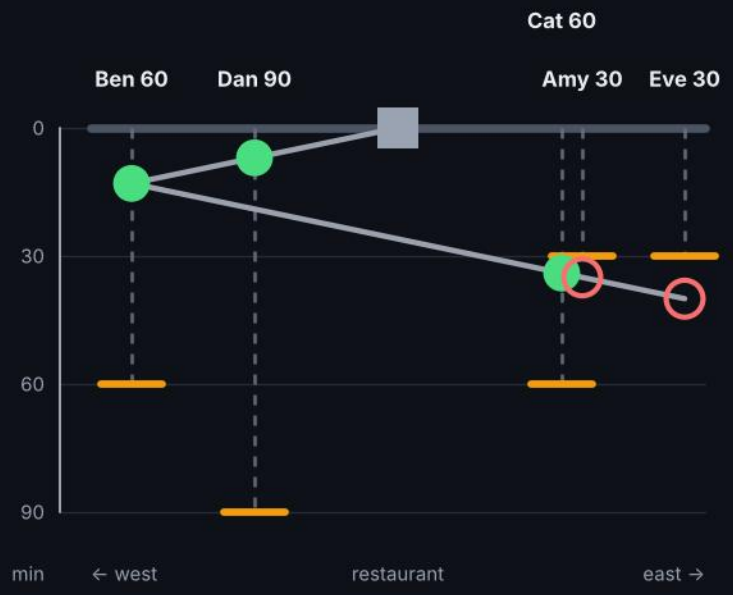
No food spoils, but rice first sent the rider the wrong way, and B never takes a choice back.

3 Evaluation

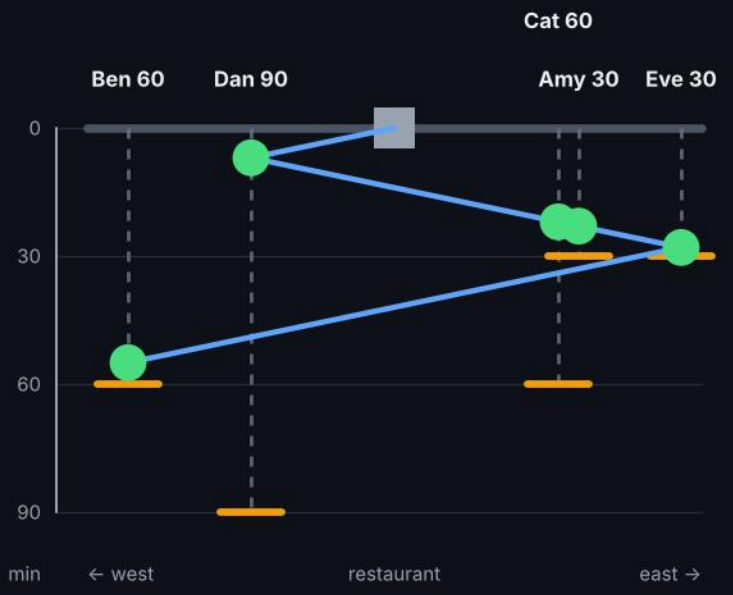
Who wins, when, and why

Head to head on the running example

Nearest neighbour 3 of 5 · 18.0 min



Approach B 5 of 5 · 27.0 min



Approach A 5 of 5 · 21.4 min



120 visiting orders



best: 21.4 min

All 120 orders checked: A's answer is the best possible here.

Each approach's trap

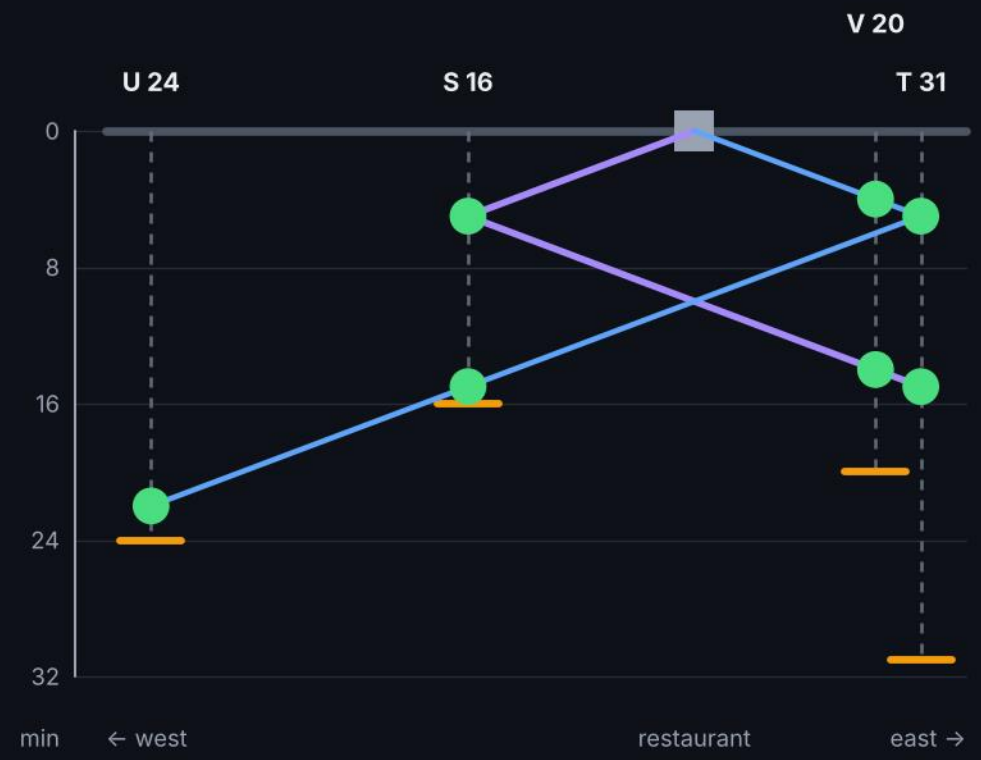
P, Q, R



B: R first (‘looks safe’), then P is lost: 2 of 3

A: P 5, R 11, Q 15: 3 of 3, avg 10.33

S, T, U, V



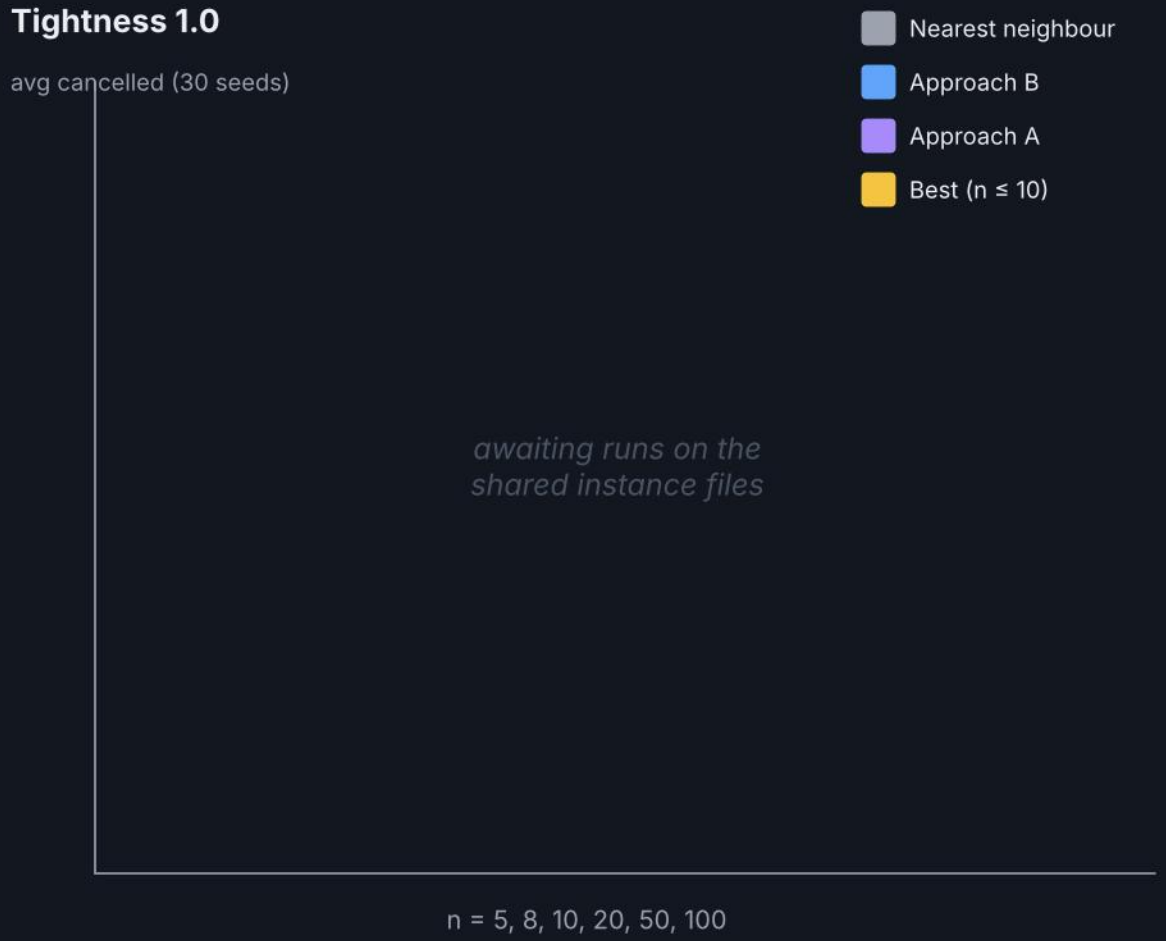
A: deadline order loses U; no single move helps: 3 of 4

B: V 4, T 5, S 15, U 22: 4 of 4

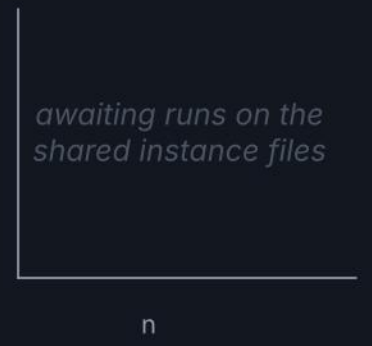
B checks orders one at a time. A changes the plan one stop at a time.

Results: orders cancelled

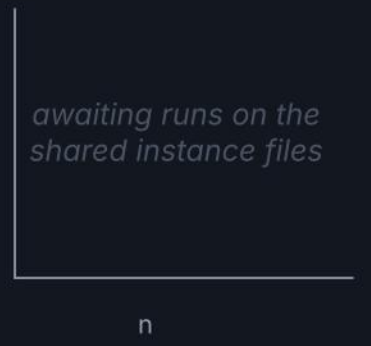
Tightness 1.0



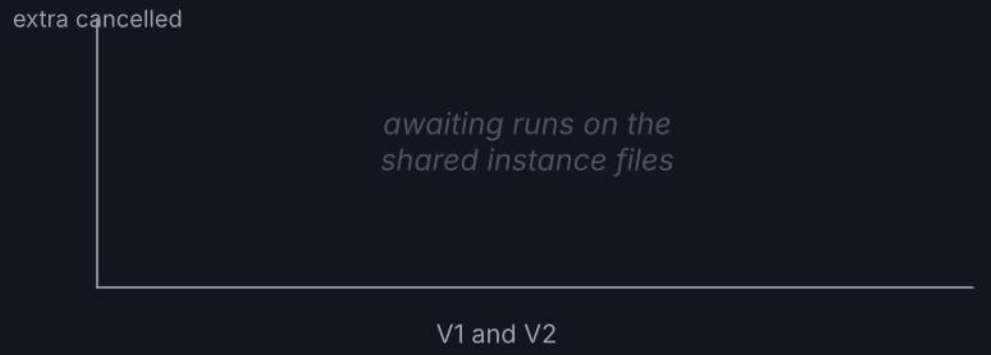
Tightness 0.5



Tightness 1.5



Gap to brute force, n ≤ 10



Results: speed



Expected: A pays for its fixes in time; B stays near-instant. (Check against the real runs.)

Complexity and trade-offs

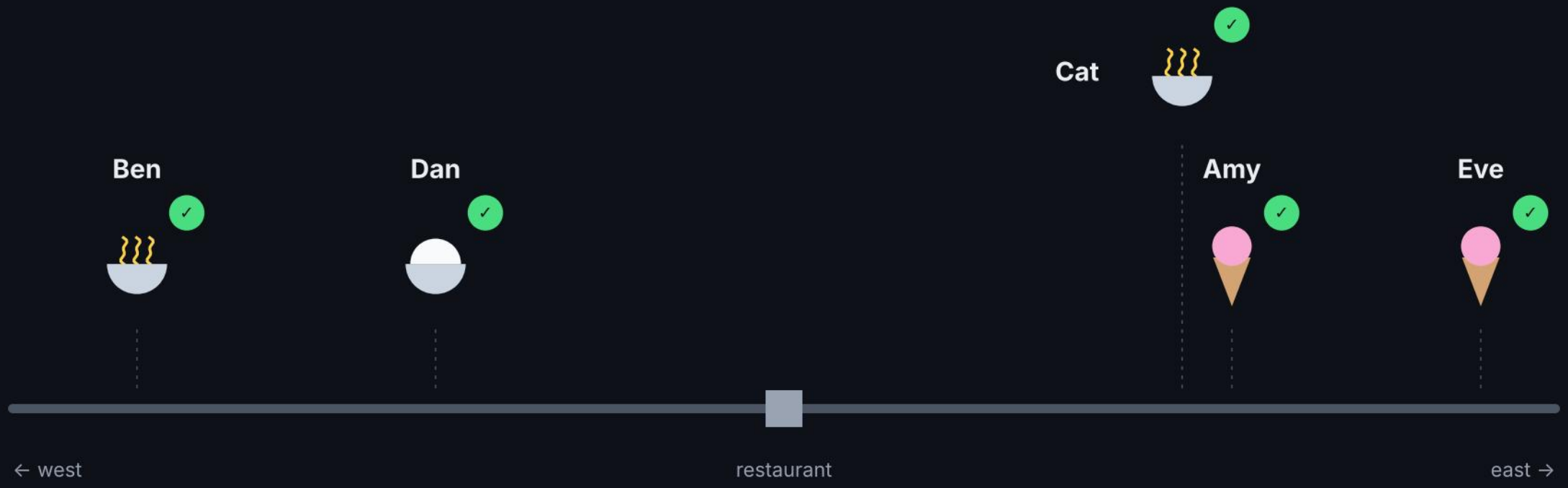
	Approach A	Approach B
Idea	Full plan, then improve	One stop at a time
Data structure	List of all n customers	visited[ ] / alive[ ] arrays
Time	$O(n \log n) + O(n^3)$ per pass × passes	$O(n^3)$, fixed
Space	$O(n)$	$O(n)$
Undo a choice / revive a cancel	Yes / yes	No / no
Trap	Needs two moves at once	Checks orders one at a time

4

Wrap-up

What we learned, and what's next

Recap



Goals in order: most on time, then least wait

Approach A

Plan first, then fix

Approach B

Decide one stop at a time

Limits and next steps

Static model

No new orders mid-route, one rider, straight-line travel times, no time at the door.

Both approaches are local

A stops at a plan no single move can improve.

B never takes a choice back.

Next

Try two-stop moves in A. Start A from B's route instead of deadline order.

Add the V2 cap to both and rerun.

What makes this stand out

A picture where ‘on time’ is visible

Position across, time down: a path above the tick is on time.

A proof that cancelling is free

Triangle inequality: once out of reach, out of reach forever.

Exact answers to grade against

All 120 orders checked by brute force; paired traps show where each approach fails.

AI usage

Tools	[e.g. Claude: which model, when]
Where	[design brainstorming · code drafting · worked examples · slides]
Our own work	[problem choice · design decisions · verification · …]

Prompt 1: [paste a representative prompt]

Prompt 2: [paste a representative prompt]

Prompt 3: [paste a representative prompt]

Thank you

Questions?



Backup: the V2 cap changes the route

Solution	Route	Cancelled	Avg wait	Longest wait
Best for V1	1→2→5→4→6→3	0	29.0	62.7 (Sentul)
Best for V2	1→2→5→3→4→6	0	29.8	55.2 (Taman Maluri)

V2 cuts Sentul's deadline from 84.7 to 54.7, so the rider goes there fourth.

A fairness cap costs 0.8 min on the average.

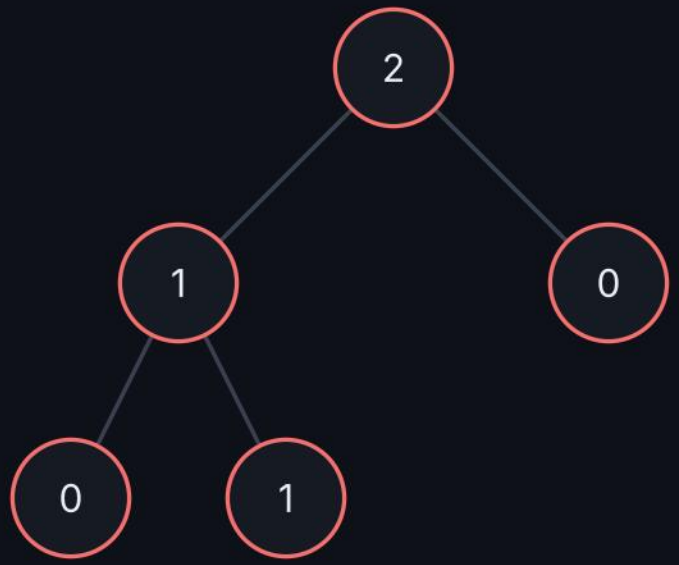
Backup: how many passes does A need?



Passes grow with n and with looser deadlines; total time = passes × $O(n^3)$.

Backup: why B has no heap

heap keyed on ‘lost’



after one move: every key changes

Rebuild every step: O(n log n)

Linear scan every step: O(n)

A's deadlines are fixed,
so sorting once is enough: O(n log n)

Backup: data limitations

- A mall stands in for the restaurant: MY.txt has no restaurant rows.
- Some coordinates are approximate: five villages share one point.
- Some points are not real customers: the city centre, the national palace, a park.
- Locations repeat above n = 33, so n = 50 and 100 are mainly speed tests.